

Linux

➤ File system hierarchy in Linux:

- / -> this is top level directory.
- /root -> it is home directory for root user. (Only Admins can have access for this)
- /home -> it is home directory for other users.
- /boot -> it contains bootable files for Linux.
- /etc -> it contains all configuration files.
- /usr -> by default software installed in this directory.
- /bin -> it contains commands used by all users.
- /sbin -> it contains commands used by super user.
- /dev -> dev directory contains the special device files for all the devices.
- /lib -> the lib folder is a library files directory which contains all helpful library files used by the system.
- /opt -> According to the Filesystem Hierarchy Standard, /opt is for "the installation of add-on application software packages".
- /tmp -> Web browsers periodically write data to the tmp directory during page views and downloads.
- /var -> Contains variable data like system logging files

➤ Linux Commands with explanation:

1. sudo

-> sudo is an abbreviation of "super user do" and is a Linux command that allows programs to be executed as a super user (aka root user) or another user.

Command: sudo

2. su

-> The purpose of sudo command is used to execute the commands with root privileges.

-> su is used to switch from one user to another

-> If no user name is specified for su, then it switches to root.

Command: su

3. whoami

-> whoami command displays the currently logged-in user.

Command: whoami

4. cd

-> cd command is used to change the current working directory

-> # indicates you are the root user

-> \$ indicates you are the normal user

Command: cd

[] cd /etc

[] cd .. -> If you want to get back to one step

[] cd ../.. -> If you want to get back two steps

5. pwd

-> pwd stands for present working directory. It will show you currently you are in which directory.

Command: pwd

6. ls -latr

-> to view all the list of files and directories we can use ls -latr

Command: ls -latr

7. Touch

-> With touch we can create empty files

Command: touch

```
[] touch (filename)
```

```
[] touch Abid
```

```
[] touch Abid1 Abid2 Abid3 -> With touch we can create multiple files at a time.
```

8. Cat

-> by using cat we create the files with data

Command: cat

```
[] cat (File Name)
```

```
[] cat Abid -> It will show the data
```

```
[] cat > Abid -> It will insert some data
```

```
[] cat >> Abid -> It will create a file and also insert some data
```

-> To save the data ctrl+d

9. Vi

-> By using vi editor or vim editor we can create files with data

Command: vi

```
[] vi (File Name)
```

```
[] vi Abid1
```

-> Press i to insert the data

-> To save the data :wq!

-> To quit the saving data :q!

10. mkdir

-> By using mkdir we can create directories

Command: mkdir

```
[] mkdir (Directory Name)
```

```
[] mkdir Abid
```

11. cp

-> cp stands for copy

-> copy command used to copy the data from one file to another

Command: cp

```
[] cp File1 File2
```

File1 = source

File2 = destination

12. mv

-> move command used to move the data from one file to another

Command: [] mv File1 File2

File1 = source

File2 = destination

13. rmdir

-> rmdir used to remove the empty directory

Command: rmdir

[] rmdir (Directory Name)

[] rmdir Abid

14. rm

-> rm is used to remove the files and directories with data.

Command: rm

[] rm -rf (File Name/Directory Name)

15. grep

-> The grep command is a filter that is used to search for lines matching a specified pattern and print the matching lines to standard output.

Command: grep

[] ls -latr |grep (File Name/Directory Name) -> To find Directory/File

[] ls -latr |grep Abid

[] Grep (word) /etc/passwd (File Name)

[] Grep Hi /etc/passwd Abid -> To find Word

16. less

-> Less command is Linux utility which can be used to read contents of text file one page (one screen) per time. It has faster access because if file is large, it don't access complete file, but access it page by page.

Command: less

[] less /etc/passwd -> Used to read contents of text file passwd

[] cd /DevOps => ls => less (File Name)

[] less Abid

17. head

-> The head command, as the name implies, print the top N number of data of the given input. By default, it prints the first 10 lines of the specified files.

Command: head

[] head /etc/passwd -> By default it will display top 10 lines

[] head -3 /etc/passwd -> It will display top 3 lines

[] cd /DevOps => ls => head (File Name)

[] head Abid

[] head -3 Abid

18. tail

-> By default tail returns the last ten lines of each file that it is given.

Command: tail

```
[] tail /etc/passwd -> By default it will display last 10 lines
```

```
[] tail -3 /etc/passwd -> It will display last 3 lines
```

```
[] cd /DevOps => ls => tail (File Name)
```

```
[] tail Abid
```

```
[] tail -3 Abid
```

19. sort

-> sort command sorts the contents of a text file, line by line.

Command: sort

```
[] sort /etc/passwd -> It will sort the contents line by line.
```

```
[] cd /DevOps => ls => sort (File Name)
```

```
[] sort Abid
```

20. clear

-> It is used to clear the terminal screen.

Command: clear

21. tree

-> Purpose: Displays directory paths and (optionally) files in each subdirectory. When you use the TREE command each directory name is displayed along with the names of any subdirectories within it.

Command: tree

```
[] yum -y install tree -> Install tree
```

```
[] tree
```

22. history

-> It will show you the history of commands you have used

Command: history

23. wget

-> wget is used to download the packages from web

Command: wget

```
[] wget urlName
```

```
[] wget http://apachemirror.wuchna.com/tomcat/tomcat-8/v8.5.54/bin/apache-tomcat-8.5.54.tar.gz
```

24. tar

-> The Linux 'tar' stands for tape archive, is used to create Archive and extract the Archive files

Command: tar

```
[] tar -xvzf filename.tar.gz
```

x: extract the archive

v: Verbose, displaying information about the archive while archiving

z: Using zip compression

f: Using an archive file

25. rpm

-> RPM command is used for installing, uninstalling, upgrading, querying, listing, and checking RPM packages on your Linux system. RPM stands for Red Hat Package Manager. With root privilege, you can use the rpm command.

Command: rpm

```
[] rpm -ivh filename.rpm
```

-i: install a package

-v: verbose

-h: print hash marks as the package archive is unpacked

```
[] rpm -e File Name -> Used to removed/uninstall package
```

26. yum

-> yum command used to install the packages

Command: yum

```
[] yum -y install python -> Used to install package
```

```
[] yum -y remove python -> Used to uninstall package
```

27. user

-> Who is using the machine.

-> A person who uses the system to use services and resources.

-> Creating user

Command: user

```
[] useradd (User Name) => useradd Abid
```

```
[] grep Abid /etc/passwd (or) cat /etc/passwd Abid (or) cd /etc => cat Abid -> Used to check user
```

```
[] passwd Abid -> Assign password to user
```

```
[] grep pavani /etc/shadow
```

28. group

-> A collection of users is called group

Command: group

```
[] groupadd (Group Name) => groupadd DevOps
```

```
[] grep DevOps /etc/group (Or) cat /etc/group DevOps (or) cd /etc => cat group -> Used to check group
```

```
[] gpasswd DevOps -> Assign password to group
```

```
[] gpasswd -M (User Name) (Group Name) => gpasswd -M Abid DevOps -> Used to add user to group
```

29. du

-> du stands for disk usage.

-> It is used to estimate file space usage.

-> The du command can be used to track the files and directories which are consuming excessive amount of space on hard disk drive.

Command: du

```
[] du
```

```
[] du -sh
```

30. df

-> Display the summary of available and used disk space usage of file system

Command: df -h

31. free

-> It shows used memory and unused memory size.

Command: free

```
[] free -m -> Used to check RAM
```

```
[] lscpu -> Used to check CPU
```

32. uptime

-> Uptime is a command that returns information about how long your system has been running.

Command: uptime

33. How to check word count in a file

-> wc command allows you to count the number of lines, words, characters, and bytes of each given file

Command: wc -c filename

34. How to find OS information

-> OS information can be found by below command

Command:

```
[] cd /etc => cat os-release
```

```
[] cat /etc/os-release
```

35. diff

-> diff is a command-line utility that allows you to compare two files line by line

Command: diff

```
[] diff file1 file2
```

36. find

-> Find command is used to search and locate the list of files and directories based on argument you have provided.

Command: find

```
[] find -name abid.txt
```

37. locate

-> locate command is used to locate the list of files and directories.

Command: locate

```
[] locate (File Name)
```

38. systemctl

-> systemctl command used to check the status of a service

Command: systemctl

```
[] yum -y install httpd -> Install httpd
```

```
[] systemctl start httpd
```

```
[] systemctl status httpd
```

```
[] systemctl stop httpd
```

39. traceroute

-> traceroute is used to know the travelling path of package through our network.

-> It will tell us package is going through machines, switches, routers.

-> It will also tell us network is working properly or not.

Command: traceroute

```
[] traceroute -n
```

```
[] traceroute google.com
```

40. **ping

-> ping stands for packet internet groper

-> It basically checks for the network connectivity between two nodes

Command: ping

```
[] ping google.com
```

41. ifconfig

-> It is used to display the ipaddress of a machine.

Command: ifconfig

42. route

-> route command is used to display the routing tabel information

-> Used to determine where data packets traveling over an Internet Protocol (IP) network will be directed

Command: route

43. hostname

-> Linux hostname is the simple command used to view and set the hostname of a system

Command: hostname

```
[] hostname
```

```
[] hostname (New Name)
```

44. netstat

-> The netstat (Network Statistics) command is a command-line tool used in Linux to monitor network connections, view routing tables, track interface statistics, and troubleshoot network issues.

Command: netstat

45. sed

->

Command: sed

```
[] sed 's/old/new/' file.txt -> Replace first match per line
```

```
[] sed 's/old/new/g' file.txt -> Replace all matches per line (global)
```

```
[] sed 's/old/new/2' file.txt -> Replace only the 2nd match per line
```

[] sed 's/old/new/gi' file.txt -> Case-insensitive, all matches

[] sed -i 's/old/new/g' file.txt -> Edit file in place

[] sed -i.bak 's/old/new/g' file.txt -> Edit in place, keep backup as file.txt.bak

[] sed -n '5p' file.txt -> Print only line 5

[] sed -n '5,10p' file.txt -> Print lines 5–10

[] sed '5d' file.txt -> Delete line 5

[] sed '/pattern/d' file.txt -> Delete lines matching pattern

[] sed '3i\text to insert' file.txt -> Insert before line 3

[] sed '3a\text to append' file.txt -> Append after line 3

GIT

Install and configure Git:

```
[] sudo su  
[] yum -y install git  
[] git --version
```

Explanation of each command with Example:

1. git config

-> Configuring username and email in git

-> We must configure username and email in git for tracking purpose

Command: git config

```
[] git config --list  
[] git config --global user.name "Abid"  
[] git config --global user.email "abid.devops@gmail.com"
```

2. git init

-> git init command is used to initialize the git repository

Command:

```
[] mkdir Abid  
[] cd Abid  
[] git init  
[] ls -la
```

3. git add

-> Create file and add that file to staging area and check the status

Command: git add

```
[] vi test1  
[] git status  
[] git add test1  
[] git add . -> For all files  
-> your files are staged now
```

4. git status

-> To view the state of the working directory.

-> Git status is to check the status of files (added or committed)

Command: git status

5. git commit

-> Commit the changes to local repo.

Command: git commit -m "commitmsg"

6. Git log:

-> git log will display the commit id and commit msg

Command: git log

[] git log -> view history of commits

[] git log --oneline -> view commit logs in oneline

7. git show

-> git show will display commitid, commitmsg and data inside the file.

-> Observer the difference between modified file and untracked file see commits id

Command: git show

8. .gitignore

-> ignore the files while committing.

-> purpose is developers are writing the code and once they build .class files also will be generated. In this case we have to eliminate .class files. By using git ignore we can eliminate .class files.

Command: .gitignore

[] vi .gitignore

> .java*

:wq!

[] git add .gitignore

[] git commit -m "ignoring .java files"

[] touch .java3 .java4

[] git status

-> here you could not see .java3 and .java4 because it is ignored by .gitignore.

-> First time only we have to commit .ignorefile. Later if any .java files are generated no need to commit. This is caused by retrospective effect.

9. Branching

-> Developers can work on their own feature branches.

Command: git branch

[] git branch -> To see list of available branches

[] git branch (Branch Name) -> To create a new branch

[] git checkout (Branch Name) -> to switch branch

[] vi c1

[] git add .

[] git commit -m "commitmsg"

-> You cannot merge branches of diff repos

-> To merge from one branch data to another branch and you have to be in destination branch

[] git merge (Branch Name) -> Used to merge branch

[] git branch -d (Branch Name) -> To delete branch

[] git branch -m (New Name) -> To rename current branch

10. Git conflicts:

-> When same file is having different contents in different branches, if you do merge then conflicts occurs.

-> Conflicts occurs in 2 ways.

1. Merging branches
2. Pushing to central repo

11. Removing files:

Command: `git rm -f (File Name)`

12. Delete all untracked files

Command:

- [] `git clean -n`
- [] `git clean -f`

13. 13. git stash

-> Removes file from working directory and puts in stashing store and gives clean working directory.

-> Developers can save their changes and come back later.

-> Later on, you can bring back that stashed items to working directory and can resume your work on that file.

-> Stash literally means save up as for future use.

Command: `git stash`

-> To stash an item

[] `vi Abid1`

[] `git add . (Or) git add Abid1`

[] `git stash`

-> Stash will be working before commit.

[] `git stash list` -> To see the stashed items list

-> To apply stashed items.

[] `git stash apply stash@{number}` -> Get back to your previous commit.

[] `git stash clear` -> To delete the stash items

14. git reset

-> To undo commit

-> It is used to bring back the file to working area.

Command: `git reset`

[] `git reset --soft (head-id)` -> will delete the commit & keep the files in staging

[] `git reset --hard (head-id)` -> bit dangerous - will delete the commit & files from workspace

[] `git reset --mixed (head-id)` -> will delete the commit & un-stage the files

15. git revert

-> A revert operation will take the specified commit, remove the changes from that commit, and create a new "revert commit".

Command: `git revert`

```
[] git revert (Commit I'D)
```

16. git remote

-> This command lets you to create, view and delete connections to other repos

Command:

```
[] git remote add origin URL
```

```
[] git remote -v -> To check the status
```

17. git clone

-> It is used to clone the remote repos

Command:

```
[] git clone URL
```

18. git fetch

-> Downloads commits, files, and references from a remote repository into your local repository without merging them into your working files

Command:

```
[] git fetch URL
```

19. git pull

-> It is used to download and merge those changes into your local repo.

-> Pull = fetch + merge

Command:

```
[] git pull origin main
```

20. git push

-> It is used to publish your local changes to your remote repo

Command:

```
[] git push origin (Branch Name)
```

21. Git tag

-> A tag is like a branch that doesn't change. Unlike branches, tags, after being created, have no further history of commits.

Tags are usually used to mark stable releases

Command: git tag

```
[] git tag
```

```
[] git tag (Tag Name2.1)
```

```
[] git push origin (Tag Name2.1)
```

22. Git cherry-pick

-> The command git cherry-pick is typically used to introduce particular commits from one branch within a repository onto a different branch.

-> Git allows you to integrate selected, individual commits from any branch into your current HEAD branch.

-> The HEAD points out the last commit in the current checkout branch. It is like a pointer to any reference. The HEAD can be understood as the "current branch." When you switch branches with 'checkout,' the HEAD is transferred to the new branch.

Command: [] git cherry-pick (Commit I'D)

23. Git fork**24. Git rebase**

-> The primary reason for rebasing is to maintain a linear project history.

25. git amend**GIT Push Solution:**

- Create a Repo
- git remote add origin URL -> Make a Connection between Repo and Local
- git clone URL
- cd -> change to Directory
- vi File Name
- git add (File Name)
- git commit -m "Commit Msg"
- git branch -> To check branch
- git branch -M main -> To switch branch
- git push -u origin main -> Push to repo
- git push origin main

-> Username:

-> Password: ghp_msk5XZO47rQPotLOGQNiFbi0KPTpuM0cqYmV

GIT Remote:

- git remote remove origin
- git remote add origin URL
- git remote
- git push -u origin main
- git push origin main

-> Username:

-> Password: ghp_msk5XZO47rQPotLOGQNiFbi0KPTpuM0cqYmV